

Premise

Biomedical knowledge graphs (KGs) integrate information across multiple ontologies to represent biological relationships (Callahan et al. 2020). Prioritizing path importance between nodes in a heterogeneous KG (multiple node types) is an open problem (Himmelstein and Baranzini 2015). Biomedical KGs exhibit a scale-free topology, characterized by a few high-degree nodes and many low-degree nodes. Shortest path search is more likely to go through higher-degree hub nodes.

High-degree nodes may genuinely display biologically important mediators in a path. However, the high degree may be an artifact of study bias or subclass-based graph structure. Paths through high-degree nodes are non-specific. Mechanistic hypotheses derived from non-specific paths are not context-specific. To identify more specific paths and metapaths, Himmelstein et al. leveraged node degree (i.e., the number of edges connecting a node) (Himmelstein and Baranzini 2015). The method employed the degree-weighted path count (DWPC) statistic based on pathway degree product (PDP) to assess metapath significance and path importance in Hetionet, a biomedical KG that the authors previously developed. Metapath DWPC values represent the sum of the PDPs of each path between two nodes of the metapath type. To adjust for higher-degree nodes, the PDP calculation raises each node degree to a negative exponent (typically -0.4 or -0.5, as seen in hetionet publications). For a source-target combination, metapaths with a higher DWPC comprise paths with nodes that have lower degrees. The path score prioritizes paths based on contribution to the metapath DWPC.

To contextualize the likelihood of observed DWPC scores, Himmelstein et al. created 200 degree-preserved permutations of the Hetionet knowledge graph (Himmelstein et al. 2023). They calculated the DWPC values for each source-target combination for each metapath. The XSWAP algorithm created permutations that preserved the degree while destroying the original graph topology (Zietz et al. 2024). The collective DWPC values of the permuted graphs for each metapath of each source-degree combination represent the effect of degree alone on DWPC values. An empirical p-value indicates the likelihood of the observed DWPC value in the context of random graphs, where edges are informed by degree but not biological associations. Significant metapaths indicate the type of information that may be helpful for understanding the source-target connection. Ideally, the results provide researchers with testable hypotheses.

The process of creating permutations and calculating DWPC values for all source-target combinations over all nodes is computationally expensive, even for smaller graphs like Hetionet. The same approach is not practical for larger knowledge graphs. The primary objective of this project is to determine whether null distributions of path counts are learnable from a single or a few permutations.

The key scientific questions are:

1. Are the null pathway predictions based on degree or other graph properties, independent (multiply individual node predictions), or conditional (the following node depends on the previous node)?
2. If independent, can a machine learning model learn to predict between the analytical prior or the empirical count prior?
3. If not independent, can a machine learning model learn to predict counts for metapath null distributions based on node degree or other pathway statistics? If so:
 - a. Which model is most accurate?
 - b. What is the minimum number of permutations necessary?
 - c. How long paths are possible to predict?

Are pathway count distributions learnable?

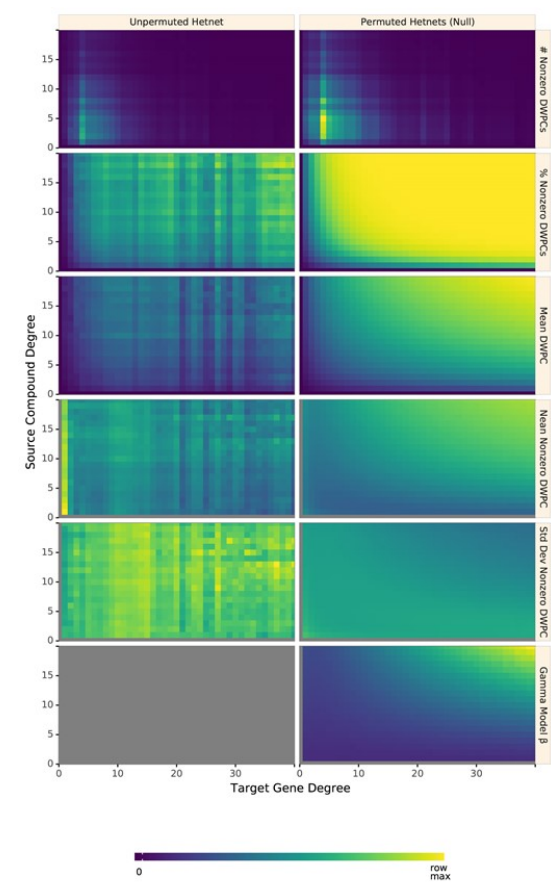


Figure 1: Caption from Himmelstein et al. (2023)
"Figure 4:
Path-based metrics vary by node degree and network permutation status. Each row shows a different metric of the DWPC distribution for the CbGpPWpG metapath—traversing Compound—binds—Gene—participates—Pathway—participates—Gene, selected for illustrative purposes. Metrics are computed for degree-groups, which is a specific pair of source degree (in this case, the source compound's count of CbG edges) and target degree (in this case, the target gene's count of GpPW edges). Metrics are reported for the unpermuted hetnet on the left and

Himmelstein et al. published Figure 1 (Himmelstein et al. 2023). This visualization shows the distribution of DWPC values for the Compound-binds-Gene-participates-Pathway-participates-Gene (CbGpPWpG) metapath by source and degree. The DWPC values for the permuted hetionets exhibit a consistent pattern: as the source and target nodes increase in degree, the DWPC values also increase. The percentages of nonzero DWPC values are significantly higher for higher-degree combinations. The mean DWPC values are also higher; however, the relationship is not as clear when the zero DWPC values are excluded.

Zero DWPC values occur when there is no path connecting source and target nodes along the metapath in the permuted network. There are numerous zero values, especially for source and target nodes with low degrees, that applying the gamma-hurdle distribution transformation helps adjust for zero inflation.

The finding that the mean non-zero DWPCs are higher for source and target degree combinations with a higher combined degree is counterintuitive, given the way the DWPC downweights node degree. The sheer number of paths still results in higher DWPC values since the DWPC is a sum.

The same pattern is not observed in the unpermuted graph. The observed DWPC value for each source-target combination is compared to the distribution of null DWPC values to calculate its significance. The unpermuted pattern looks to be largely driven by the target degree. The high std dev of the unpermuted data indicates that there must be a biological signal within the source-target degree combinations beyond the graph artifacts alone.

Figure 2 shows count statistics for the same CbGpPWpG metapath in 5 graphs permuted in 2025 (notebooks 0-1). The total path counts panel on the top left represents the mean path count per pair for each degree combination. The observed pattern corresponds to the pattern from 2023 over all 200 permutations seen in Figure 1.

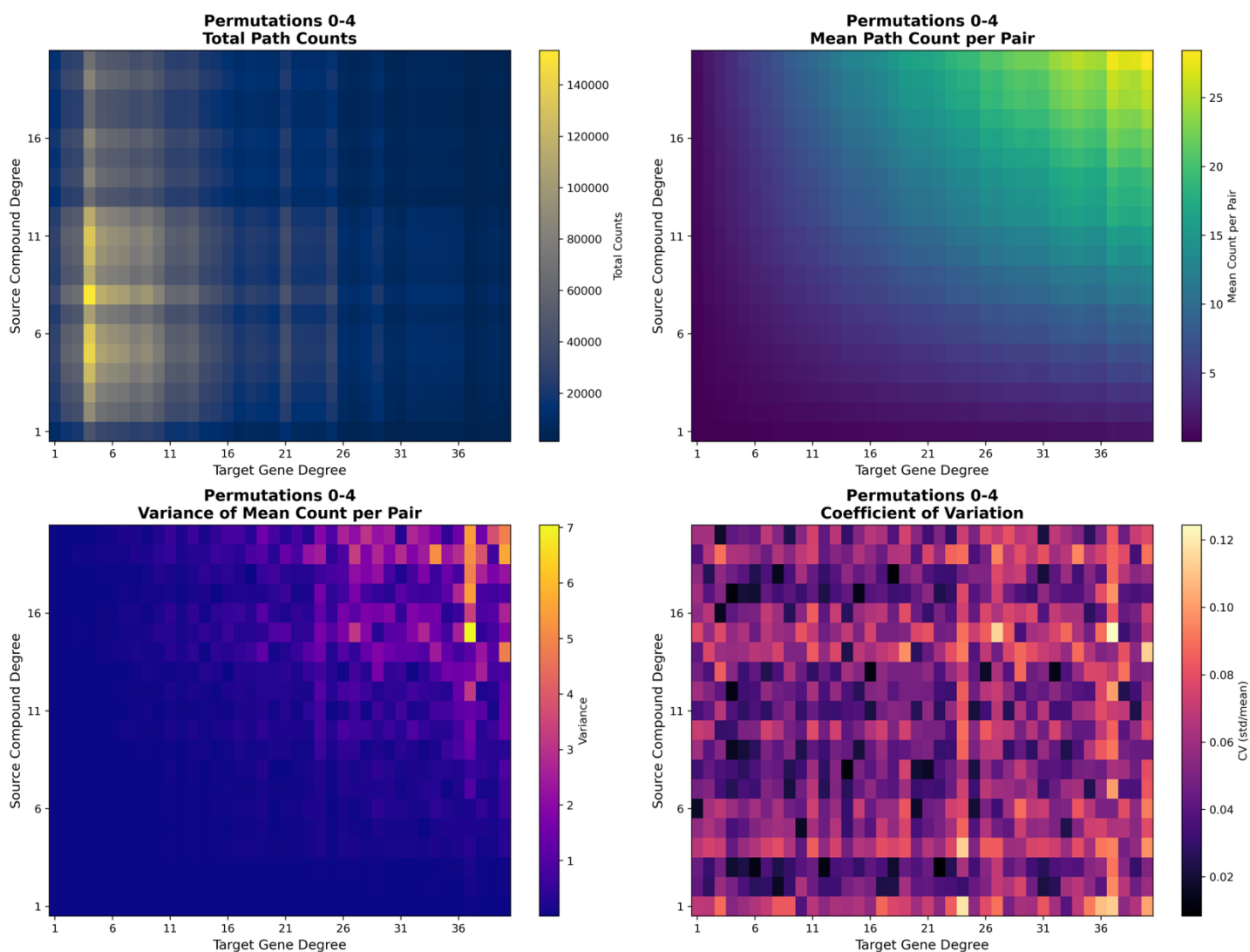


Figure 2: Heatmaps of count statistics from 5 permutations for the CbGpPWpG metapath. The top-left plot shows the sum of the mean counts for each source-target degree combination. The top right plot shows the mean of the mean counts. The bottom left and right plots show the variance and coefficient of variation, respectively, of mean counts over the 5 permutations. These figures were created with `create_path_count_heatmap.py` and saved as `results/path_count_visualization/CbGpPWpG_path_count_heatmap.png`.

The top right panel shows the mean of the mean counts for each source-target degree pair. The pattern matches the mean nonzero DWPC and gamma model β from 2023. The number of pairs decreases as the source-degree product increases. For the total counts this results in higher counts for lower degrees. However, source-degree combinations with higher degrees have more paths between them. This is the pattern that the DWPC null comparison corrects for. The bottom left plot shows that the variance increases with higher degrees, showing that there is more variability in path counts as node degree increases. However, the bottom right shows uniform coefficient of variance, which demonstrates that the relative variability is consistent across degree ranges.

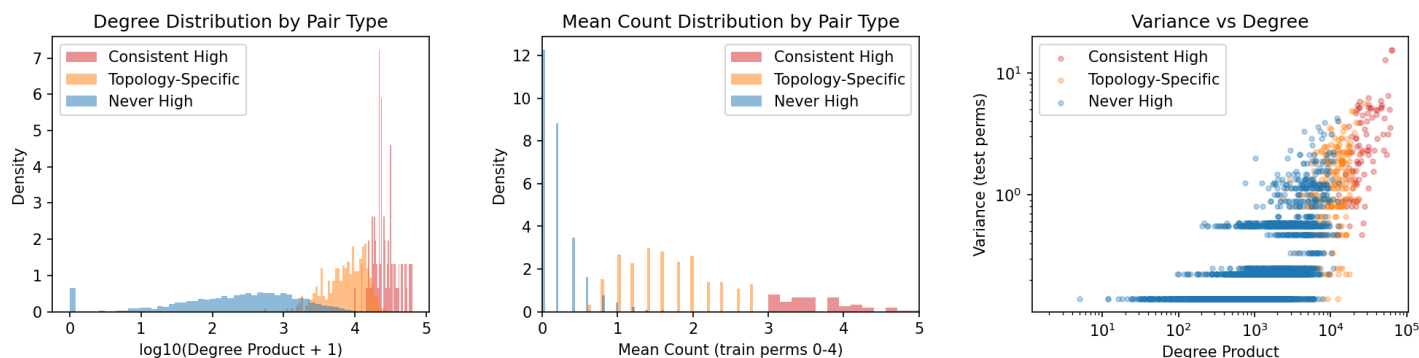


Figure 3: Count by degree exploration for CbGpPWbG. For all 3 plots, blue are pairs that never have high counts over permutations 0-4, orange are pairs with high counts only in specific permutations, and red includes pairs with consistently high counts across permutations. The left plot shows the density of nodes by degree product. The middle shows the count distribution by pair type. The right plot shows the variance by degree. This plot was created with `visualize_model_failures.py` and output as `results/model_failures/model_failure_analysis.png`.

Figure 3 visualizes the predicted counts for CbGpPWbG across permutations 0-4, classifying node pairs as having never high counts, topology-specific counts where the counts vary greatly between different permutations, and consistently high counts. As seen in Figure 2, there is increased variance in counts for high degree product pairs when compared to the test set of 5 permutations. However, this plot does not include the relative error or coefficient of variation, which may show more consistent or higher variability in the topology-specific pairs, as in the bottom right panel of Figure 2. Taken together, Figures 2 and 3 demonstrate that absolute variance of path counts increases as degree product increases.

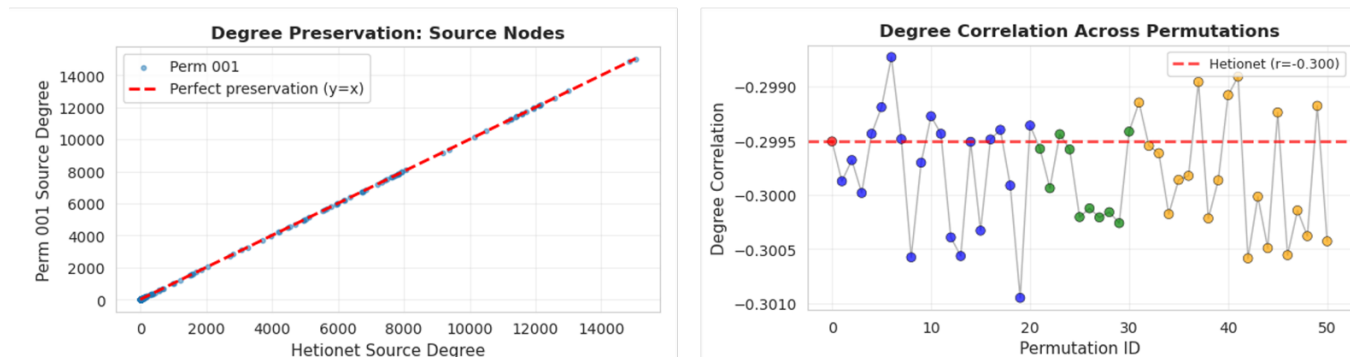


Figure 4: Permutation similarity investigation. The left panel shows the source degree relationship between hetionet and a permuted graph. The right panel shows the edge correlation for AeG across 50 permutations. Hetionet is the red dot. These plots are created using `notebooks/permutation_similarity_analysis.ipynb` and located in `results/permutations_similarity/AeG_permutation_similarity.png`.

An investigation of permutation similarity revealed that XSWAP effectively preserves the node degrees (Figure 4, left panel) and node degree correlations for edges (Figure 4, right panel). Figure 4 used the edge AeG as an example, where the genes have lower edge-specific degrees than anatomy nodes, resulting in the negative correlation pattern. Thus, the XSWAP algorithm maintains the degree properties, yet show high variability across permutations in path topology, especially for higher degree product pairs where there are consistently more paths.

The patterns observed in the mean of mean path counts, the null DWPC values, and the consistent coefficient of variation across degree combinations indicate the potential for learning the null path counts. However, the increased variance in higher degree combinations may affect the generalizability of predictions across permutations for those pairs. This variability exists in the 200 empirical null graphs computed for the 2023

hetionet publication. The variability is captured in the empirical null DWPC calculations. However, the DWPC priors capture the variance for each pair as a distribution of DWPC values for comparison. **How do the means of these priors relate to the counts by degree pair? Would using the mean and variance of counts or predicted counts adequately capture the null distribution for disease pairs?**

Are the null pathway predictions independent or conditional?

In an independent model, pathway counts are the product of individual edge counts (or probabilities). In a conditional model, the presence of the next edge is dependent on the properties or annotations of the previous node. The actual hetionet graph will demonstrate a conditional pathway relationship since the nodes contain biological meaning. However, there is a question as to whether the null pathways also exhibit this relationship. For a high-fidelity independent pathway null, it is necessary to have accurate edge counts or predictions. As this project aims to avoid the creation of multiple permutations, the ability to predict edge counts was evaluated.

Can a machine learning model learn to predict more accurately than the analytical prior or the empirical count prior?

Testing this question required a close reading of Zeitz et al. (2024), where the authors used a modified XSWAP algorithm to create permuted bipartite graphs. For comparison, the authors derived an analytical prior to predict the probability of edge presence based on the source-target node degrees:

$$P_{i,j} = \frac{u_i v_j}{\sqrt{(u_i v_j)^2 + (m - u_i - v_j + 1)^2}}$$

where u_i and v_j are the source and target degrees, respectively, and m is the total number of edges in the network. With the goal of predicting the edge frequencies with a high correlations to the empirical frequencies observed in the 200 permutations for the 2023 himmelstein paper, the analytical prior was compared against predictions using a simple neural net, logistic regression, polynomial logistic regression, and random forest models. The models were trained on individual bipartite graphs for each edge type to predict edge presence for each based on source degree and target degree.

The analytical prior performed well regardless of edge density (Figure 5). Random forest and polynomial logistic regression models performed the best of the tested machine learning models. Surprisingly, the neural net model performed poor for each edge density with a high variance of correlations. This relationship may be due to problems in calibrating the neural net predictions. In Figure 6, the Simple NN panel indicates that the model is learning a functional relationship effectively; however, this relationship is not well calibrated to the empirical frequencies. Another interesting observation is that the other machine learning models, as well as the analytical prior, show strange striations in the empirical frequencies for the predictions. In this case the model is underpredicting for certain degree combinations, especially for those where there is a low source degree and high target degree (Figure 7). The model performance did not change greatly when adding more permutations (notebooks 6,7).

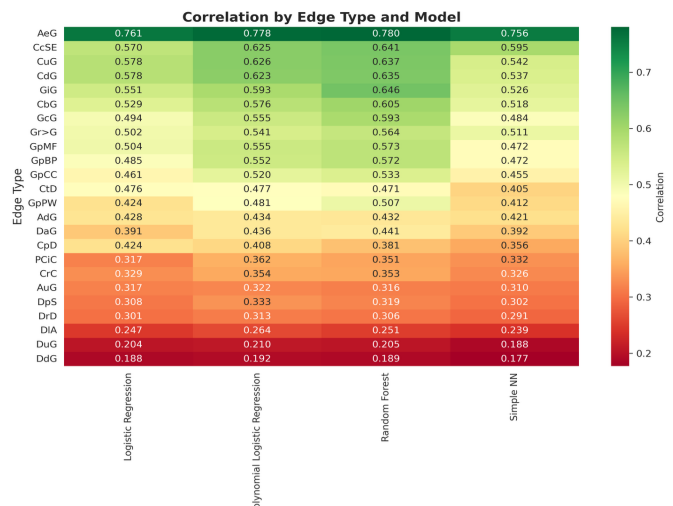
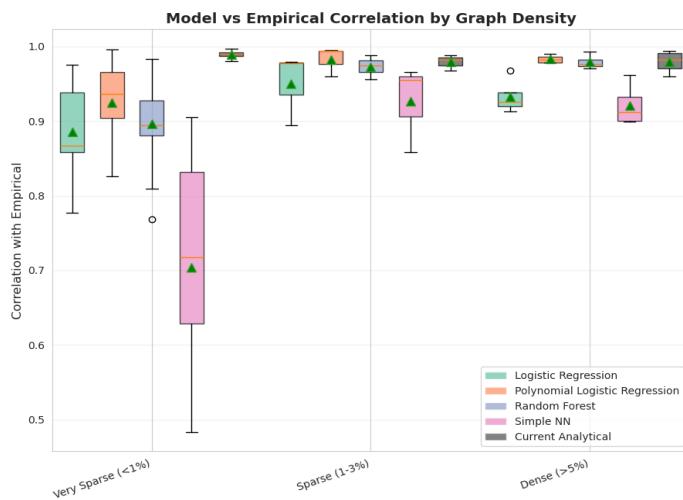


Figure 5: Summary of model correlations over edge types. The left panel separates edges by graph density and shows correlations across the models tested and the analytical prior. The right plot shows the correlation across edges for the models tested. These plots are created by running notebook 4 over all edges and notebook 5 to summarize the results.

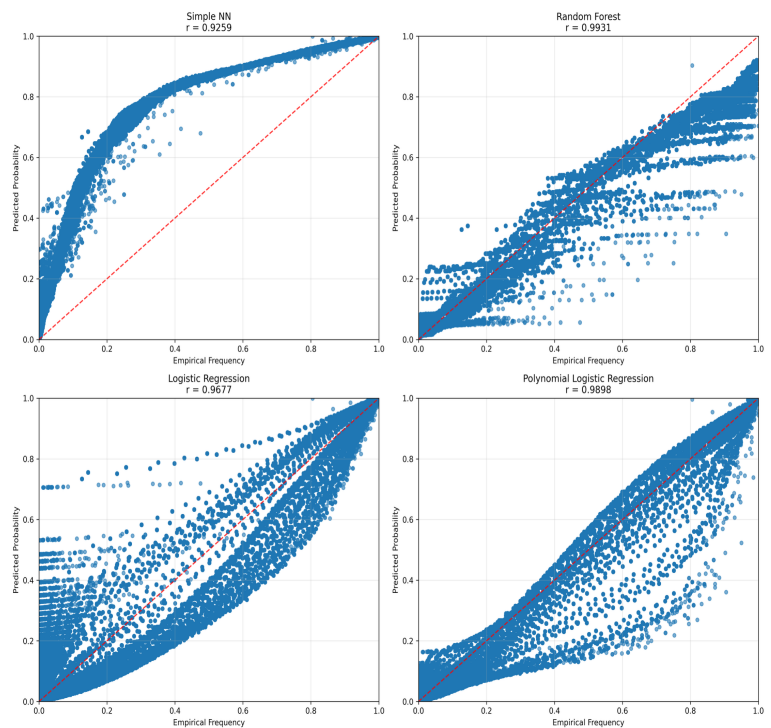
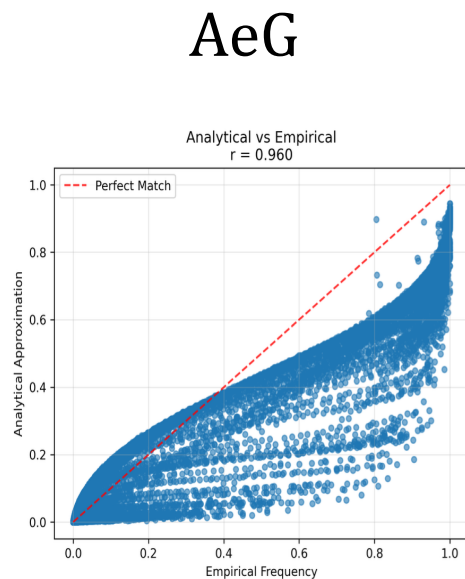


Figure 6: Comparisons between model predictions and empirical frequencies. These plots were created by notebook 4.

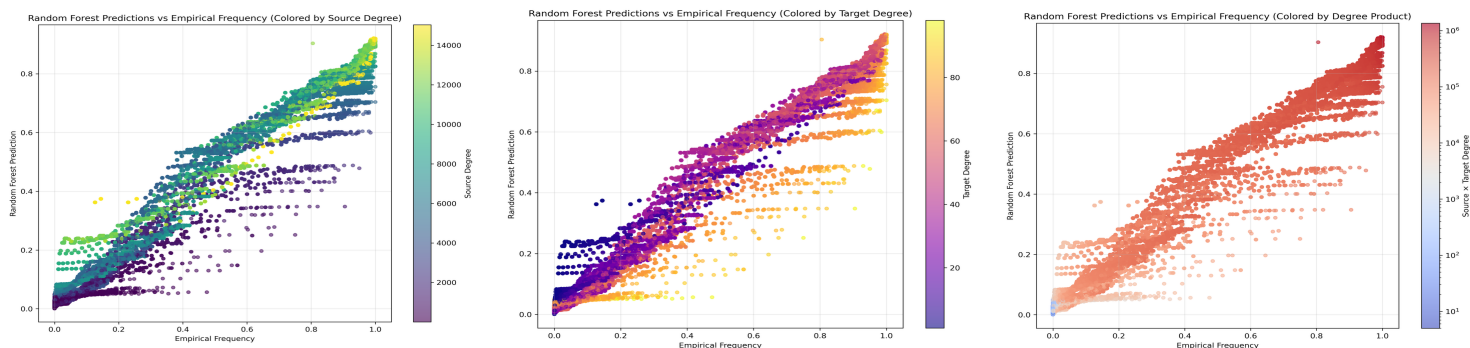


Figure 7: Degree patterns as related to random forest prediction error for the AeG edge. These plots are computed in notebook 4.

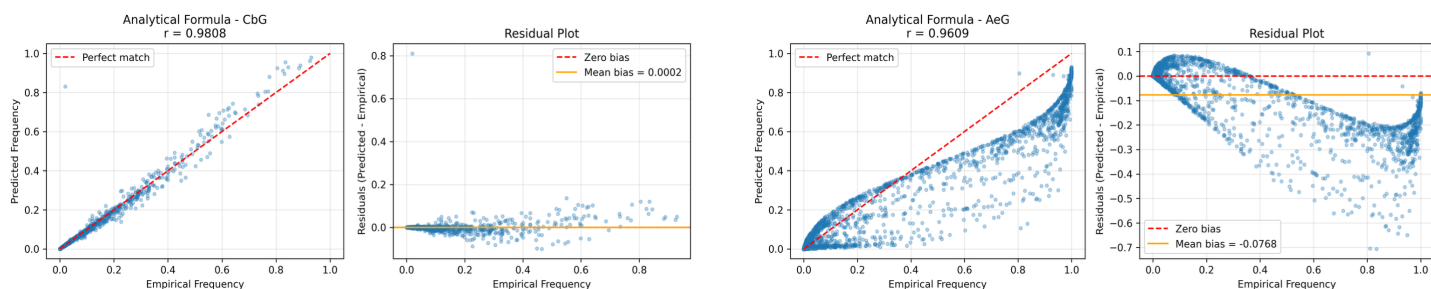


Figure 8: Residual bias of analytical prior for CbG and AeG. These plots were created in notebook 8 with degree analysis.

Overall, the analytical does consistently well across edge types. However, the residual bias is not consistent across edge types (Figure 8). To address this bias, several experiments tested the incorporation of learnable parameters into the analytical prior formula, including polynomial and density-based features (notebooks 8-9). For some edge types, the learned models reduced the mean absolute error yet reduced the model correlation (Figure 9). It is clear from looking at the AeG edge plots that while the points are closer to the expected trend line, there is still consistent bias in prediction across edge frequencies. Due to the reduced correlation and the relatively consistent performance, this line of testing was abandoned.

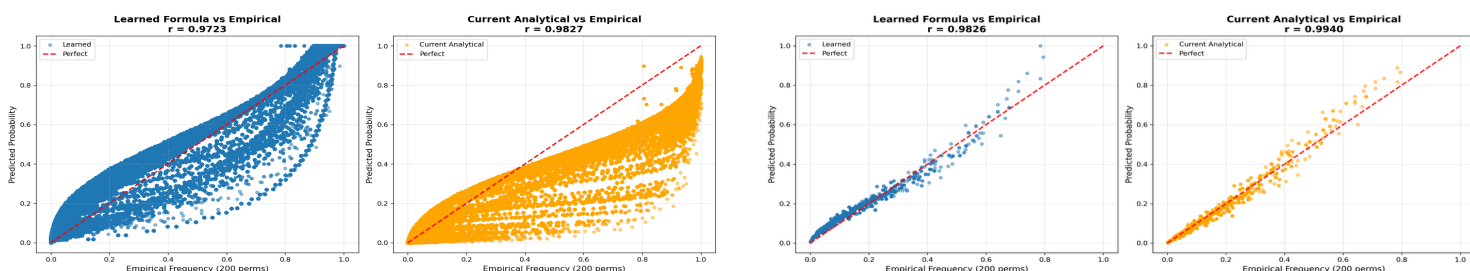


Figure 9: Comparisons between the analytical formula learned with updated parameters. These figures were created by notebook 8.

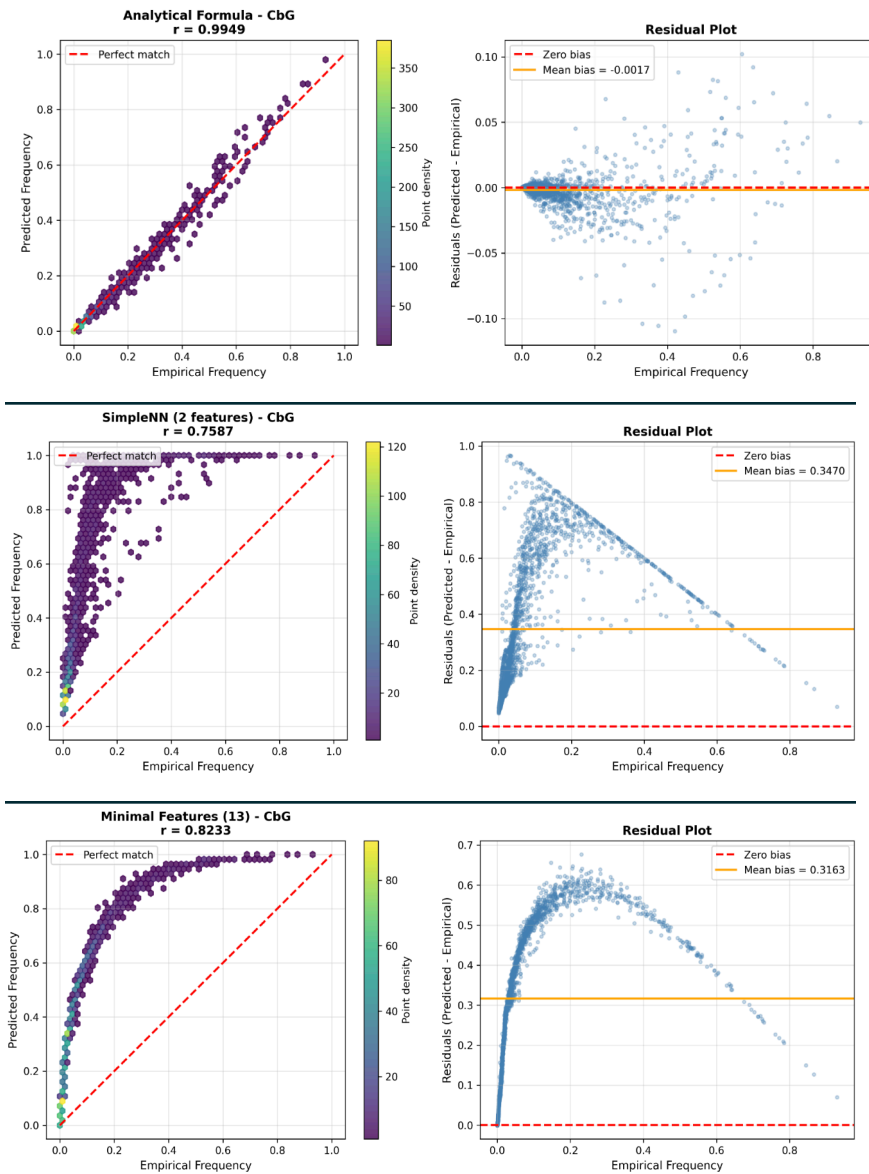


Figure 10: Analytical prior and NN correlation and residual plots. These plots were created by notebook 21.

The neural net models consistently demonstrated residual bias, specifically positive bias, regardless of whether the model used a simple NN with only node degree as input features or multiple features that capture the nonlinear effects of degree combinations (Figure 10). Other neural net approaches using different optimizers (L-BFGS or Adam), different input feature sets of node properties (e.g., assortativity, centrality), different hidden layer sizes, different prediction functions, or different architectures (e.g., GNN, CNN) yielded similar results. Similarly, incorporating degree-aware corrections, degree binning, or regularization yielded similar results. These tests were performed in notebooks 13,14, and with several scripts:

- phase1_baseline_model.py - Baseline pair-level model
- run_pair_level_phase2.py - Degree-aware corrections
- test_phase3_feature_comparison.py - Feature representation comparison
- test_phase3_binning_resolution.py - Binning strategy testing
- test_phase4_control_experiments.py - Control experiments
- test_phase5_linear_regression_cv.py - Linear regression with CV
- test_phase5b_bias_diagnostics.py - Bias analysis
- test_phase5b_theoretical_correction.py - Theoretical corrections
- test_phase5b_degree_aware_correction.py - Degree-aware corrections
- test_phase5c_regularization.py - Regularization testing

Overall, the machine learning models did as well or worse than the analytical prior. Therefore, the analytical prior was used for testing the independent or conditional compositional nulls.

Can an independent pathway model based on multiplying analytical priors accurately predict path frequency?

The probability of a path existing between a source and target node in the CbGpPW pathway was tested using

$$P(c \rightarrow p) = 1 - \prod_{g \in G_{c,p}} [1 - P(c \rightarrow g) \cdot P(g \rightarrow p)]$$

where the probabilities are based on the analytical prior over permutations 0-19. The predictions were evaluated against the actual frequencies observed in permutations 20-29, and with the pairwise mutual information, where

$$PMI = \log_2 \left(\frac{\text{observed frequency}}{\text{predicted frequency}} \right).$$

The top panels of Figure 11 show that the pathway frequency predictions were not very accurate. ($r \sim .35$). The model underpredicted the frequency of many pairs and overpredicted the frequency of others. The overlapping histograms of PMI scores show that the independent model predicted similarly using permutation data as with the hetionet graph data. Since the permutations are degree invariant and the independent model is based on combining edge probabilities based on degree, this behavior is expected. Investigating the poor prediction of the model by degree combinations indicated that the model predicted more for combinations between very sparse nodes than very high nodes. Based on the differences in degree PMI, degree binned independent models were tested over multiple metapaths. The source binned model performed better than the unbinned (overall) model, however the target binned model performed worse. Despite these differences, it is hard to draw conclusions about path structure since the model still performed poorly. Source-target product binned independent models were not tested.

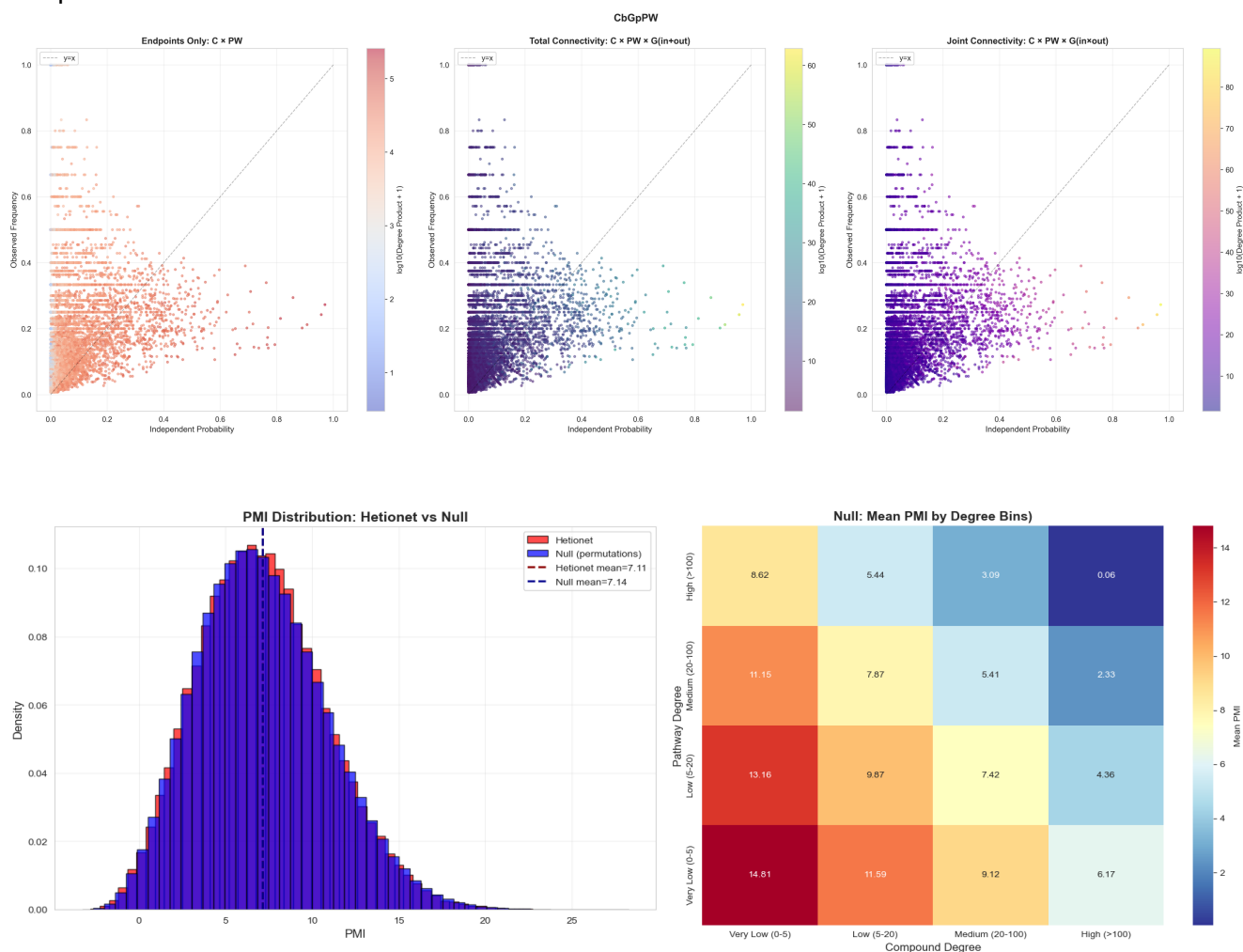


Figure 11: Results from the independent pathway prediction models. The top panels show predicted frequencies in comparison to the observed frequencies. The overlapping histograms demonstrate similar PMI between the prediction models based on permutation data to the actual data. The heatmap on the right breaks down the PMI performance by the sparsity (ie., degree) of source and target nodes. These plots were created in notebooks 11.2 and 11.3.

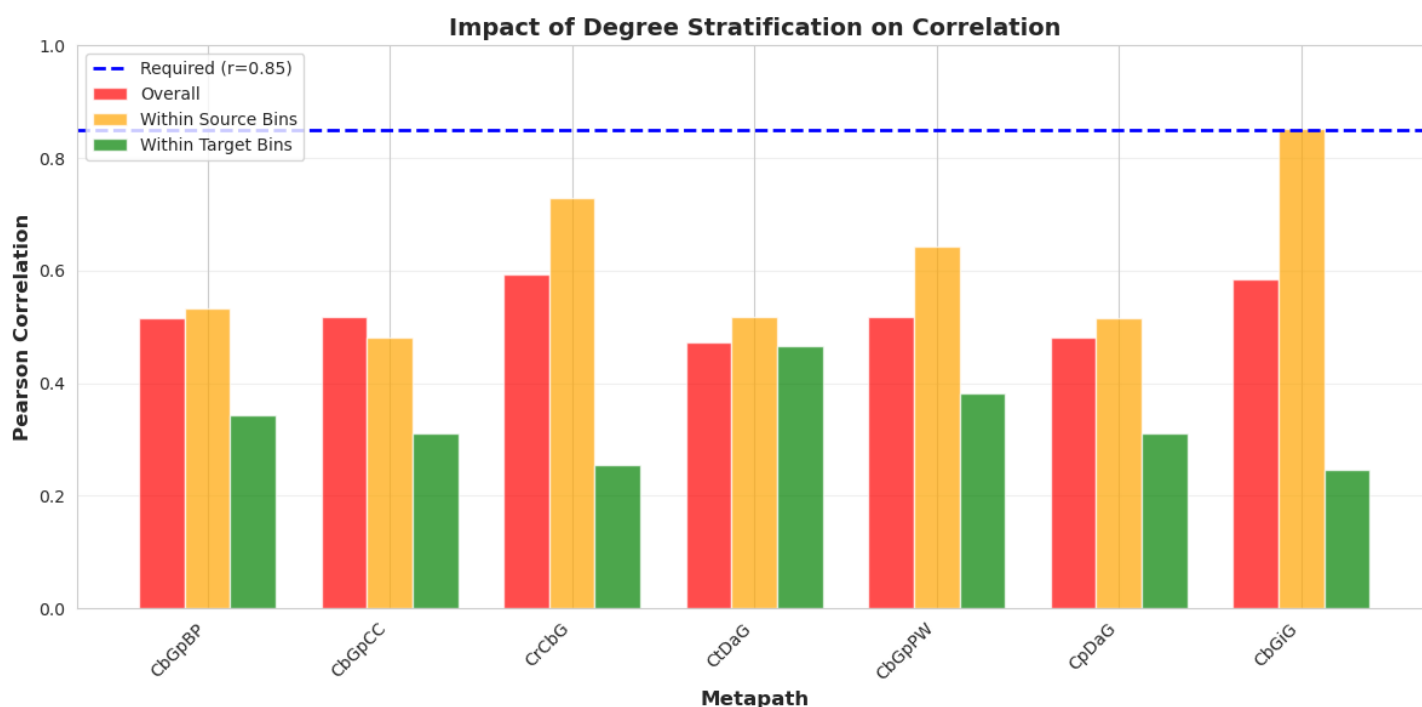


Figure 12: Correlations using source or target degree bins over multiple 2-hop metapaths. This figure was created with Notebook 12.

The improved independent model performance for higher degree combinations (Figure 11) seemed to contradict the observed increase in variance for higher degree combinations (Figures 2 and 3). The top left panel of figure 13 shows PMI over source and target degree in higher resolution revealed that the model performs best for nodes with very low or very high degree combinations. Along those lines, the middle panel of Figure 13 demonstrates that PMI is low or high variance node combinations. As figure 2 shows the low variance paths are the low degree combinations while the high variance nodes are the high degree combinations (Figure 13, top right panel). However, high-degree nodes exhibit lower coefficients of variation, resulting in a lower PMI. This behavior may relate to the law of large numbers, where more intermediate nodes in high degree paths lead to more averaging reducing fluctuations. Or, in the context of high fluctuations the independence assumption accounts for this noise. However, the middle bottom panel shows that there are several nodes with medium high counts that have high PMI (worse alignment to the actual counts), which may point to more topology-based effects not accounted for by degree alone.

Several other notebooks and scripts tested the independence assumption in various ways, including:

1. 11_degree_conditioned_compositionality.ipynb - Production analysis, tests PMI
2. 11.1_empirical_vs_analytical_compositional.ipynb - Old broken method (deprecated)
3. 11.2_empirical_vs_analytical_compositional.ipynb - Creates your plot, Option A method
4. 11.3_empirical_vs_analytical_compositional.ipynb - Full 20-perm analysis
5. 10_metapath_compositionality_analysis.ipynb - Core compositional test
6. 12_degree_aware_compositional_model.ipynb - Degree-stratified improvements
7. 14_fast_compositional_null.ipynb - Null model training
8. 15_metapath_null_distributions.ipynb - Null distributions
9. 17_compositional_validation.ipynb - Tests accuracy (FAILED: $r=0.35$)
10. 17b_compositional_failure_analysis.ipynb - Analyzes why it fails
11. test_src/test_phase4_control_experiments.py
12. test_src/test_hierarchical_subpath_exp2a.py
13. test_src/test_hierarchical_degrees_exp2b_optimized.py
14. test_src/debug_option_a.py

Taken together, the independent model did not adequately capture pathway count prediction and thus conditional models were tested.

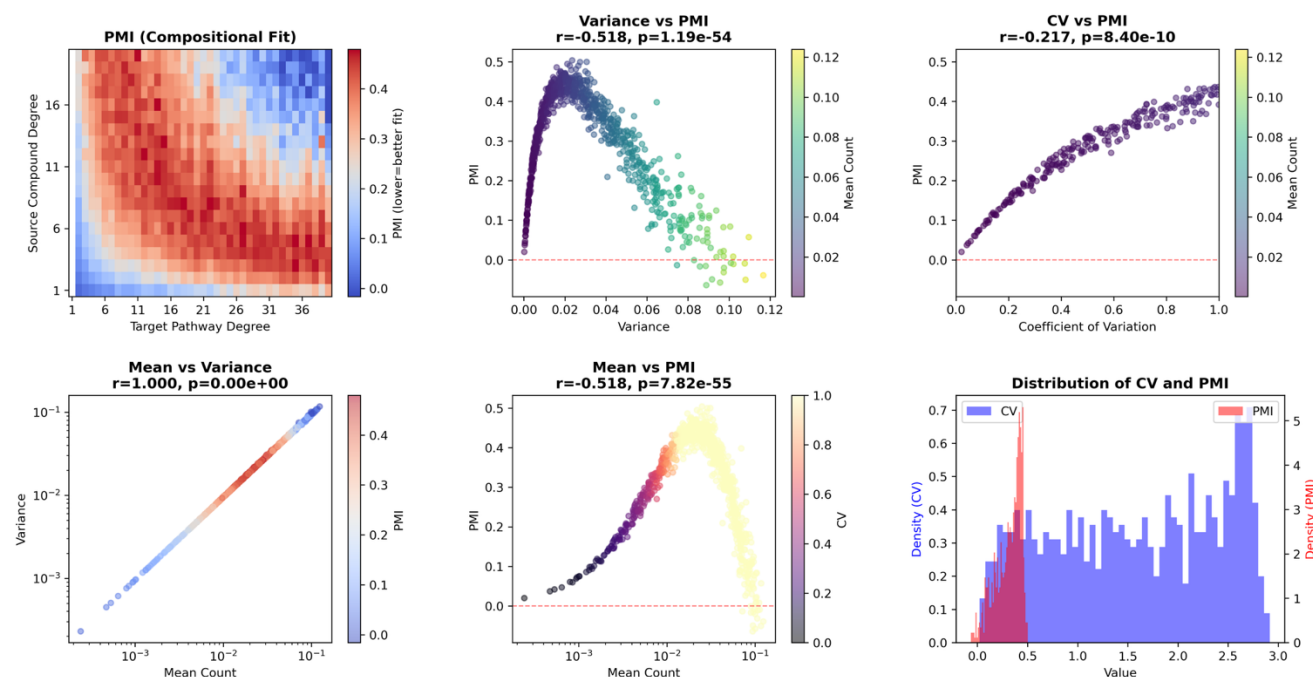


Figure 13 Variance to PMI relationship. The top plots show PMI over source node degree combinations, Variance vs PMI, CV vs PMI. The bottom row shows Mean count vs variance, Mean count vs PMI, Distribution comparison: CV and PMI (this one is probably not helpful). These plots are created with the `analyze_variance_pmi_relationship.py`.

Can a conditional model accurately predict null pathway counts?

The conditional model assumes that the previous node or nodes in a pathway affect the next node. In other words, the next node is conditioned on the previous nodes. Since the null distribution breaks down the topology of the graph while maintaining degree characteristics, the original hypothesis was that an independent model would account for the null paths. Nevertheless, experiments with the null model did not support this hypothesis.

Notebooks 18-22 tested the conditional model through direct pathway prediction using various neural network configurations and comparison models (RF, GLM) trained on degree information. The notebooks in the 18 series showed promising results, but the experimental design had data leakage between the training and testing datasets. Notebooks 19-21 corrected this experimental design and found that using an MSE loss weighted function performed best, with degree binning performed well. However, there was a substantial variation within degree bins. Moreover, the prediction task involved the mean counts of the paths across multiple held-out permutations based on a single permutation. Degree binning using a single permutations and evaluation on permutation means was not rigorously capturing the variation across permutations for the null distribution (Figure

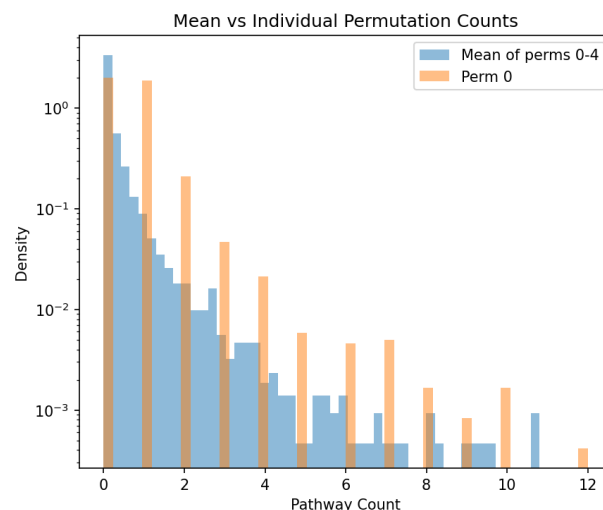


Figure 14: Pathway counts in perm 0 vs. the average for permutations 0-4. This figure was generated using `test_topology_specific_outliers.py`

14). Therefore, the next phase of experiments validated the mean count predictions against held-out permutation counts individually.

For the experiments evaluating performance over individual permutations, several models were tested, including: GLM, NegBin GLM, polynomial glm, gradient boosting, NN, and a GNN, Bayesian hierarchical models, and multi-task learning. These tests were completed with scripts:

1. test_gnn_pathway_counts.py - Test GNN to break $r=0.787$ ceiling
2. test_nonlinear_mean_models.py - Compare RF, GB, NN, Poly, NegBin GLM
3. test_bayesian_hierarchical.py - Bayesian hierarchical model
4. test_multitask_metapaths.py - Multitask learning across metapaths

Table 1: Count prediction performance for CbGpPW

Model	Test_r	Delta_vs_Baseline_pct	QQ_Corr
Baseline Linear	0.787	0	0.71
Bayesian Hierarchical	0.785	-0.3	0.779
Gradient Boosting	0.78	-0.9	0.792
Random Forest	0.778	-1.1	0.815
Heteroscedastic NN	0.777	-1.3	0.816
Multi-Task Learning	0.764	-2.9	
GNN Single-Perm	0.757	-3.8	
Polynomial deg-3	0.755	-4.1	0.874
Weighted Grad Boost	0.769	-2.3	0.691
NegBin GLM	0.743	-5.6	0.749
GNN Multi-Perm	0.743	-5.6	
Weighted RF	0.776	-1.4	0.719
Neural Network	0.759	-3.6	0.774
Log Features	0.54	-31.4	0.949

The results of the model testing are reported in Table 1. Surprisingly, the models all performed similarly and outperformed the benchmark based on the log feature. The models varied in their calibration to the permutations, as evidenced by the differences in QQ correlation. For the tested models, there does not seem to be a clear trade-off between the correlation and QQ correlation; however, this is seen for the log features. This pattern is also present in some longer paths (see below). In general, the models exhibited a similar failure pattern, characterized by higher variance and consistently high path counts across permutations (Figures 15 and 16). Thus, there is a substantial unexplained variance from the topological differences across permutations for higher degree node combinations. Also, the heteroskedastic NN, which attempts to correct for the heteroskedasticity, shows very poorly calibrated count predictions (in fact, it looks to be predicting negative counts!!). In summary, it appears that the models perform similarly, thus the computationally efficient models (linear, RF) provide an advantage.

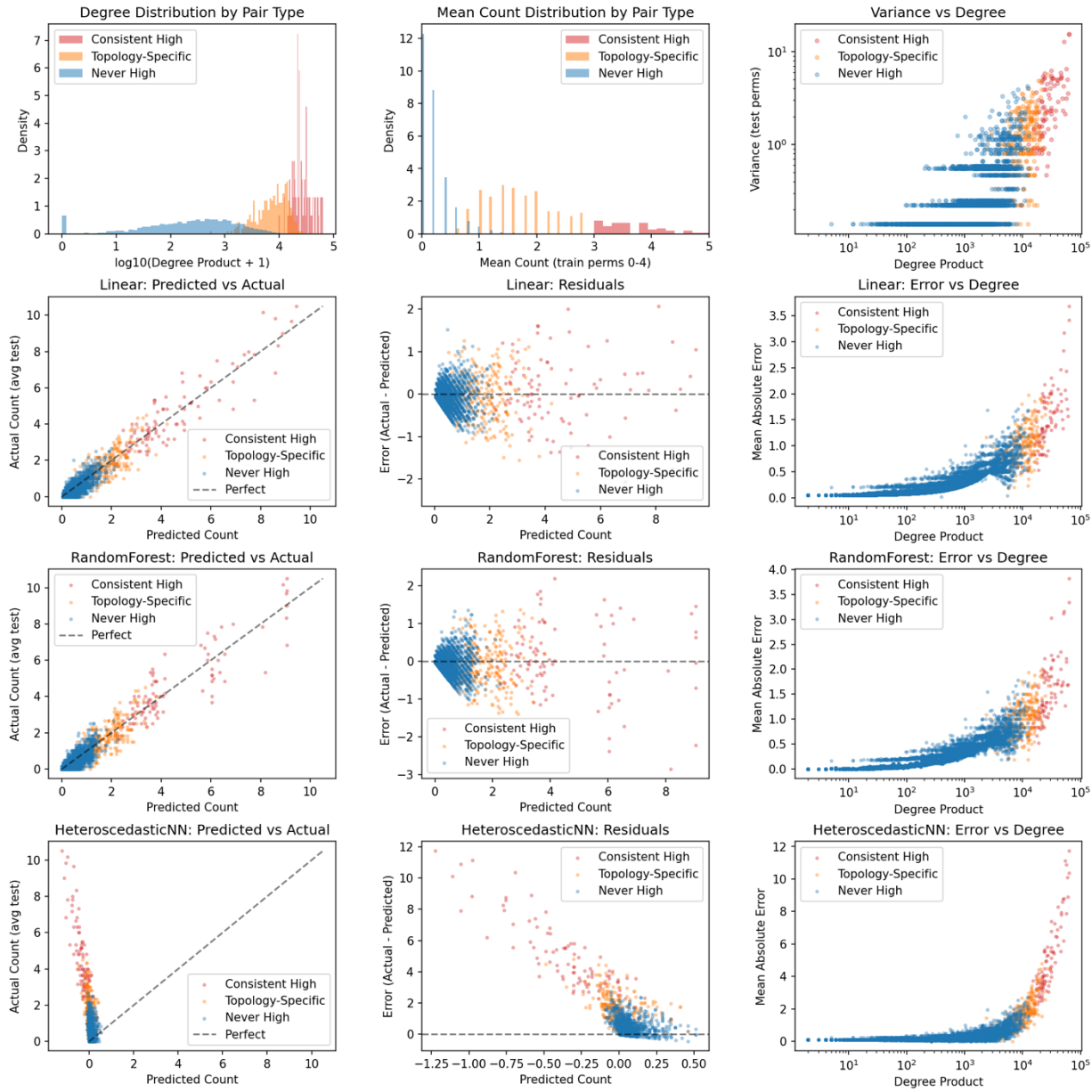


Figure 15: Count by degree exploration for CbGpPWbG. For all plots, blue are pairs that never have high counts over permutations 0-4, orange are pairs with high counts only in specific permutations, and red includes pairs with consistently high counts across permutations. This plot was created with visualize_model_failures.py and output as results/model_failures/model_failure_analysis.png.

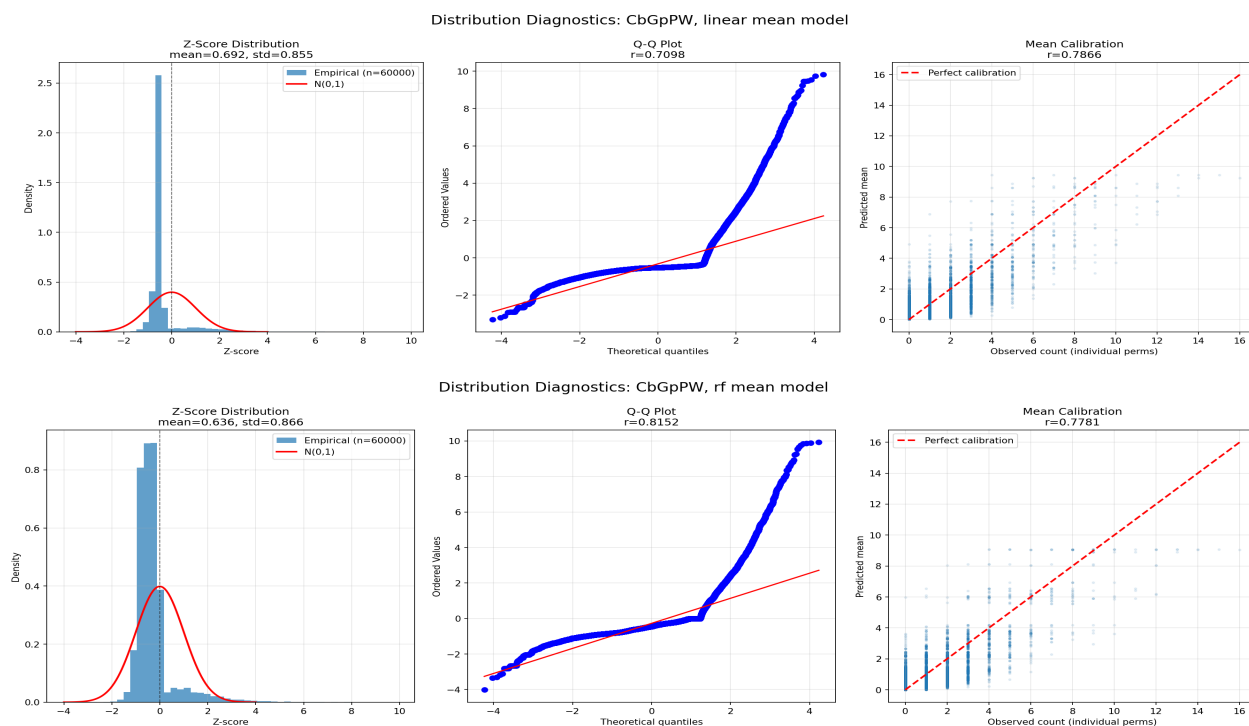


Figure 16: Shows the distribution of Z-scores across multiple permutations, the QQ plot of the Z-score correlations, and the mean correlation of predictions. These figures are created with `test_nonlinear_mean_models.py`.

The prediction of paths longer than 3 hops (more than 4 nodes) included a similar count prediction to the shorter paths. First, tests evaluated whether a hierarchical, semi-independent model where the predictions from shorter paths were aggregated. These tests were performed with:

1. `test_degree_stratified_composition_exp2f.py` - Degree-stratified composition
2. `test_focused_composition_exp2g.py` - Focused composition with sparsity
3. `test_analytical_composition_exp2h.py` - Analytical prior composition
4. `analyze_exp2g_sparsity_effect.py` - Analyze sparsity effect

The semi-independent models seemed to train well ($r > .8$), yet the predictions failed to generalize across permutations ($r < .6$).

Direct path prediction for longer paths was tested with the following scripts:

1. `test_length4_paths.py` - Test length-4 paths (5 metapaths)
2. `test_length4_endpoint_only.py` - Endpoint-only for length-4
3. `test_length5_endpoint_only.py` - Endpoint-only for length-5
4. `test_length_degradation.py` - Performance across lengths 2-8

These tests showed that the model correlation improved at longer path lengths, but the calibration deteriorated and reached the lowest point at length 5 (Figure 17). This is related to the total path count and mean counts per pair (Table 2). As the path length increases, the mean paths per pair is greater, which leads to less overall variance in prediction. The dip at length 5 shows that

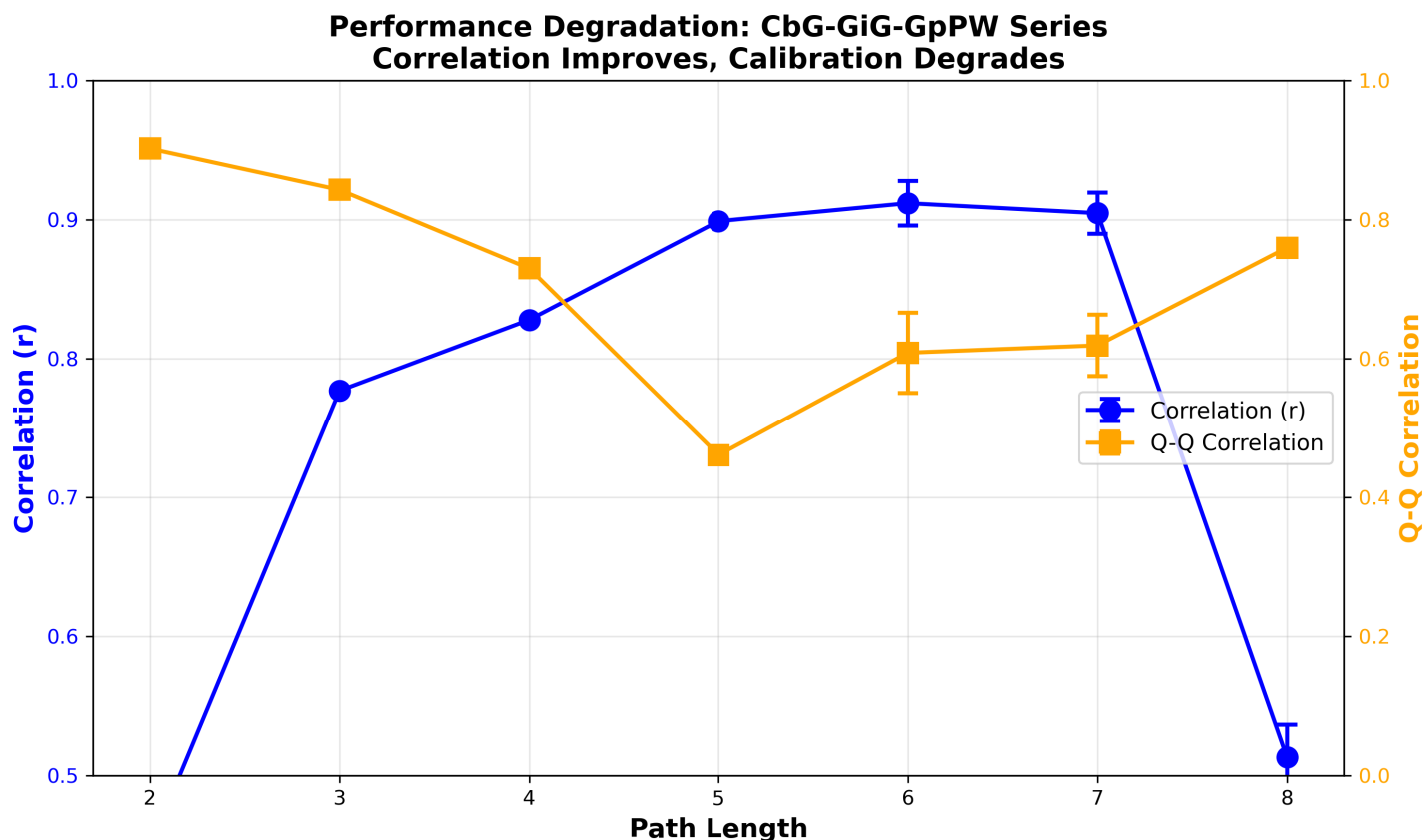


Figure 17. Shows performance over longer paths. Created with `test_length_degradation.py`

path_length	metapath	total_path_count	mean_paths_per_pair	sparsity
2	CbG	11571	1	0.99964404
3	CbG-GpPW	149545	1.21850759	0.95659862
4	CbG-GiG-GpPW	4867627	4.5957038	0.62543674
5	CbG-GiG-GiG-GpPW	698077744	286.501944	0.13833961
6	CbG-GiG-GiG-GiG-GpPW	72711729742	29155.9373	0.11806337
7	CbG-GiG-GiG-GiG-GiG-GpPW	8.99111E+12	3604636.76	0.11791237
8	CbG-GiG-GiG-GiG-GiG-GiG-GpPW	5.34447E+14	214265619	0.11791237

Are the hetionet p-values statistically valid?

The high variance of path counts across permutations for nodes with high degree combinations would indicate broad distributions of DWPC values in the null distributions. Intuitively, this would make it hard to find a significant path in the Hetionet graph. Moreover, the increased variance for longer paths (paths of length ≥ 5) would also display a wide distribution, especially for high degree nodes. Nevertheless, both high degree and long paths are significant by the Hetionet connectivity search calculation. This raises the hypothesis that the reported p-values may in some way violate statistical assumptions.

To test this hypothesis, an experiment was performed calculating DWPC p-values for a single permutation with a typical null DWPC background (based either on 20 newly permuted permutations or the 200 permutations from the Himmelstein et al. (2023) publication. The expectation was that, for the permutation where the topological signal is destroyed, the p-values are uniform across the product of source and target degrees. As a positive control, these p-values are compared to p-values from Hetionet.

References

- Callahan, Tiffany J., Ignacio J. Tripodi, Harrison Pielke-Lombardo, and Lawrence E. Hunter. 2020. "Knowledge-Based Biomedical Data Science." *Annual Review of Biomedical Data Science* 3 (1): 23–41. <https://doi.org/10.1146/annurev-biodatasci-010820-091627>.
- Himmelstein, Daniel S., and Sergio E. Baranzini. 2015. "Heterogeneous Network Edge Prediction: A Data Integration Approach to Prioritize Disease-Associated Genes." *PLOS Computational Biology* 11 (7): e1004259. <https://doi.org/10.1371/journal.pcbi.1004259>.
- Himmelstein, Daniel S, Michael Zietz, Vincent Rubinetti, et al. 2023. "Hetnet Connectivity Search Provides Rapid Insights into How Biomedical Entities Are Related." *GigaScience* 12 (January): giad047. <https://doi.org/10.1093/gigascience/giad047>.
- Zietz, Michael, Daniel S Himmelstein, Kyle Kloster, Christopher Williams, Michael W Nagle, and Casey S Greene. 2024. "The Probability of Edge Existence Due to Node Degree: A Baseline for Network-Based Predictions." *GigaScience* 13 (January): giae001. <https://doi.org/10.1093/gigascience/giae001>.